

BronchoTrack_V3

Fusion Package -- Pseudocode Reference

Prepared by	Aveesha Nishendra
Date	September 02, 2026
Scope	bronchotrack/fusion/ (9 files), bronchotrack/scalar_kalman.py, tests/test_fusion_smoke.py

This document lays out pseudocode for every file in the **bronchotrack.fusion** package -- the three-model fused localization pipeline (diameter-growth motion model, point-based bearing matching, and whole-mask ratio method, combined through a Kalman filter) -- plus the shared **scalar_kalman.py** filter it depends on and its smoke-test suite. Each entry gives the file's role in one line, followed by a functional walkthrough of its classes and functions in the order they execute.

bronchotrack/scalar_kalman.py

Shared 2-state constant-velocity Kalman filter ([value, rate]), reused by both paper_exact (roll angle, diameter:distance smoothing) and fusion (diameter growth rate, per-label confidence smoothing).

```
function build_matrices(dt):
    F = [[1, dt], [0, 1]]      # state transition: value += rate*dt
    H = [[1, 0]]              # we only observe "value", not "rate"
    return F, H

class ScalarKalmanFilter:
    state: [value, rate], covariance P, process_noise Q, measurement_noise R

    function predict(dt):
        F, _ = build_matrices(dt)
        state = F . state
        P = F . P . F_transpose + Q
        return self

    function update(measurement):
        F, H = build_matrices(dt=1)
        y = measurement - H . state          # innovation
        S = H . P . H_transpose + R
        K = P . H_transpose . inverse(S)      # Kalman gain
        state = state + K . y
        P = (I - K . H) . P
        return self

    property value: state[0]
    property rate: state[1]
```

bronchotrack/fusion/motion_model.py

Diameter-growth motion model: tracks each tracklet's apparent diameter (Kalman-smoothed) and derives a scale-free time-to-contact. Cross-checked (never calibrated) against the airway graph's known branch lengths.

```
class ApproachEstimate:
    fields: track_id, diameter, growth_rate, approaching (bool), tau_frames (nullable)
    function detection_diameter(tracklet):
```

```

    return tracklet's box height      # proxy for apparent lumen size

class ApproachMotionModel:
    filters: map of track_id -> ScalarKalmanFilter    # smooths diameter + its rate of change

    function update(tracklets, frame_idx):
        estimates = {}
        for t in tracklets:
            diam = detection_diameter(t)
            filt = filters.get(t.track_id) or new ScalarKalmanFilter(seeded at diam)
            filt.predict(dt=1); filt.update(diam)
            filters[t.track_id] = filt

            growth_rate = filt.rate
            approaching = growth_rate > small_epsilon    # only "approaching" if genuinely growing
            tau_frames = filt.value / growth_rate if approaching else None
            # tau = diameter / d(diameter)/dt -- scale-free time-to-contact
            estimates[t.track_id] = ApproachEstimate(t.track_id, filt.value, growth_rate,
                                                    approaching, tau_frames)

        return estimates

    function tau_frames_for(track_id):
        return estimates.get(track_id).tau_frames if present else None

    function prune(live_track_ids):
        remove any entry in filters whose track_id is not in live_track_ids

class TransitRecord:
    fields: label, frames_spent, arc_length_mm

class TransitSanityChecker:
    plausible_range: (low, high)    # mm per frame bounds
    history: list of TransitRecord

    function record_transit(label, frames_spent, arc_length_mm):
        append TransitRecord to history

    property median_speed_mm_per_frame:
        return median(arc_length_mm / frames_spent for each record), or None if empty

    function check(tau_frames, candidate_arc_length_mm):
        if history is empty: return "unknown"
        implied_speed = candidate_arc_length_mm / tau_frames
        ratio = implied_speed / median_speed_mm_per_frame
        return "plausible" if plausible_range.low <= ratio <= plausible_range.high
            else "implausible"    # empirical, self-referential check, not a calibration

```

bronchotrack/fusion/bronchotrack_id.py

'The bronchotrack version': point-based bearing matching using each lumen's segmentation center, re-derived fresh rather than importing paper_exact.association's implementation.

```

function detection_point(tracklet):
    return polygon_opposing_center(tracklet.mask) if tracklet has a mask
        else tracklet's box center

function bearing_cost_matrix(graph, ref_label, candidate_labels, detections, reference_point):
    if candidate_labels is empty: return None
    projected = graph.project_children_2d(ref_label)    # 2D unit bearing per candidate child
    if any candidate label is missing a projection: return None

```

```

cost = matrix[len(candidate_labels)][len(detections)]
for i, label in candidate_labels:
    for j, det in detections:
        observed_bearing = normalize(detection_point(det) - reference_point)
        cost[i][j] = 1 - cosine_similarity(observed_bearing, projected[label])
return cost      # 0 = perfect bearing match, up to 2 = opposite direction

```

bronchotrack/fusion/ratio_id.py

'The ratio method': genuinely whole-mask, not point-based. Size is the mask's own area-equivalent diameter; distance is the nearest boundary-to-boundary gap between two full mask polygons.

```

function mask_boundary_gap(poly_a, poly_b):
    if either polygon is None: return None
    return min distance over all point pairs (p in poly_a, q in poly_b)
    # nearest boundary-to-boundary gap, an all-pairs search over both boundaries

function circle_boundary_gap(center_a, radius_a, center_b, radius_b):
    d = distance(center_a, center_b) - radius_a - radius_b
    return max(d, MIN_DIST)  # clamp so overlapping circles never go to zero/negative

function reference_opening_radius(graph, ref_label):
    return graph node ref_label's known physical radius, or None if graph has no radius data

function ratio_cost_matrix(graph, ref_label, candidate_labels, reference_detection,
                           detections, virtual_advance_mm):
    ref_radius = reference_opening_radius(graph, ref_label)
    if ref_radius is None or detections is empty or reference_detection.mask is None:
        return None

    ref_equiv_diameter = mask_equivalent_diameter(reference_detection.mask)
    # area-based (shoelace integral over the mask boundary), not longest-diameter
    px_per_mm = ref_equiv_diameter / (2 * ref_radius)

    cost = matrix[len(candidate_labels)][len(detections)]
    for i, label in candidate_labels:
        child_radius = graph.child_apparent_equivalent_diameter_at_distance(
            ref_label, label, virtual_advance_mm) / 2
        for j, det in detections:
            if det.mask is None: cost[i][j] = large_penalty; continue
            gap_px = mask_boundary_gap(reference_detection.mask, det.mask)
            gap_mm = gap_px / px_per_mm
            expected_gap_mm = circle_boundary_gap(using ref_radius and child_radius)
            cost[i][j] = abs(gap_mm - expected_gap_mm)
            # how well this detection's real-world spacing matches the graph's expectation
    return cost

```

bronchotrack/fusion/kalman_fusion.py

Blends the bearing and ratio cost matrices into one Hungarian assignment per frame, nudges the result by the motion model, and smooths each candidate's confidence with a per-label Kalman filter.

```

class CandidateConfidence:
    fields: label, fused_confidence, matched_track_id, raw_confidence

class IdentityFusion:
    bearing_weight, ratio_weight, motion_gate_strength
    per_label_filters: map of label -> ScalarKalmanFilter  # smooths confidence over time
    function update(labels, detections, bearing_cost, ratio_cost, approach_estimates):

```

```

blended_cost = blend(bearing_cost, ratio_cost, bearing_weight, ratio_weight)
# if one cue is missing (None), fall back to using only the other, unweighted

row_idx, col_idx = hungarian_assignment(blended_cost)
# one detection per candidate label, minimizing total cost

results = {}
for each (label, detection) matched by the assignment:
    raw_confidence = 1 - blended_cost[label][detection]
    gated_confidence = apply_motion_gate(raw_confidence,
                                         approach_estimates.get(detection.track_id))

    filt = per_label_filters.get(label) or new ScalarKalmanFilter
    filt.predict(dt=1); filt.update(gated_confidence)
    fused = filt.value

    results[label] = CandidateConfidence(label, fused, detection.track_id, raw_confidence)
return results

function apply_motion_gate(confidence, estimate):
    if estimate is None: return confidence
    return confidence + motion_gate_strength if estimate.approaching
    else confidence - motion_gate_strength

```

bronchotrack/fusion/association.py

End-to-end continuous re-evaluation: re-runs the fused match every frame against whatever is currently visible, and commits a branch transition only after a consecutive-frame confidence streak.

```

class AssociationResult:
    fields: location, committed_this_frame, transition_sanity, candidates

function detection_size(tracklet):
    same as motion_model's diameter proxy (box height)

class FusionAssociation:
    graph, commit_threshold, commit_frames
    current_location, has_committed_once = False
    commit_streak: map of label -> consecutive-frame count
    motion_model = ApproachMotionModel()
    sanity_checker = TransitSanityChecker()
    fusion_by_location: map of location -> IdentityFusion # one fusion instance per branch point
    eligibility_fn: always returns True (no tracker dropout in this design)

function process_frame(tracklets, frame_idx):
    approach_estimates = motion_model.update(tracklets, frame_idx)
    motion_model.prune(live track ids)

    if not has_committed_once:
        return try_bootstrap(tracklets, frame_idx)

    reference = pick_reference(tracklets)
    # the tracklet currently labeled as occupying current_location
    if reference is None:
        return AssociationResult(current_location, committed=False,
                                sanity="unknown", candidates={})

    children = graph's children of current_location
    fusion = get_fusion(current_location) # lazily created IdentityFusion per location
    bearing_cost = bearing_cost_matrix(graph, current_location, children,

```

```

        tracklets, detection_point(reference))
ratio_cost = ratio_cost_matrix(graph, current_location, children, reference,
                               tracklets, virtual_advance_mm)
candidates = fusion.update(children, tracklets, bearing_cost, ratio_cost,
                           approach_estimates)

committed, sanity_verdict = try_commit(candidates, tracklets, frame_idx)
return AssociationResult(current_location, committed, sanity_verdict or "unknown",
                        candidates)

function pick_reference(tracklets):
    return the tracklet whose .label == current_location, if any, else None

function try_bootstrap(tracklets, frame_idx):
    # special one-time case: at the very first bifurcation there is no
    # pre-existing occupant, so both visible lumens are labeled from
    # bearing alone in a single shot
    children = graph's children of the graph root
    match each visible tracklet to the nearest child bearing
    (bearing_cost_matrix only, no ratio/motion yet)
    label every matched tracklet accordingly
    current_location = label of the larger / most prominent matched tracklet
    has_committed_once = True
    return AssociationResult(current_location, committed=True,
                            sanity="unknown", candidates={})

function try_commit(candidates, tracklets, frame_idx):
    for label, cand in candidates:
        if cand.fused_confidence >= commit_threshold:
            commit_streak[label] += 1
        else:
            commit_streak[label] = 0      # reset immediately, not decayed

    best_label = candidate with highest streak that has reached commit_frames
    if no such label: return (False, None)

    matched_tracklet = tracklet with track_id == candidates[best_label].matched_track_id
    arc_length_mm = graph distance from current_location to best_label
    tau = motion_model.tau_frames_for(matched_tracklet.track_id)
    sanity = sanity_checker.check(tau, arc_length_mm) if tau is not None else "unknown"
    if sanity resolves and location actually changes:
        sanity_checker.record_transit(best_label, frames_spent=commit_streak[best_label],
                                    arc_length_mm)

    current_location = best_label
    reset commit_streak for all labels
    update_gallery(matched_tracklet, best_label)
    return (True, sanity)

function update_gallery(tracklet, label):
    tracklet.label = label      # so pick_reference finds it next frame

function generation(label):
    return graph node's generation depth (used for HUD / logging)

```

bronchotrack/fusion/pipeline.py

Drives detector, tracker, and association together frame by frame, writes the overlay video and the JSON log.

```

class FrameResult:
    fields: frame_idx, location, committed_this_frame, candidates, tracklets

```

```

class FusionPipeline:
    detector, tracker, association: FusionAssociation, graph

    function process_frame(frame_bgr, frame_idx):
        detections = detector.infer(frame_bgr)
        active_tracklets = tracker.update(detections, frame_idx,
                                          eligibility_fn=association.eligibility_fn)
        result = association.process_frame(active_tracklets, frame_idx)
        return frame_log_entry(frame_idx, result, active_tracklets)

    function run_on_video(video_path, out_video_path, out_json_path):
        open video reader, video writer
        log = []
        for each frame, frame_idx in video:
            result = process_frame(frame, frame_idx)
            overlay_frame = viz.draw_overlay(frame, result, graph)
            write overlay_frame to out_video
            log.append(result)
        write_json_log(log, out_json_path)
        close reader/writer

    function frame_log_entry(frame_idx, result, tracklets):
        return dict with frame_idx, current location, committed flag,
            per-candidate confidences, per-tracklet box/label info

    function write_json_log(log, path):
        serialize log list to JSON file

```

bronchotrack/fusion/viz.py

Draws the live overlay: tracked lumens plus a HUD panel listing every currently scored candidate child branch (not fabricated screen positions, since no camera calibration exists in this pipeline).

```

function draw_overlay(frame_bgr, result, graph):
    out = copy of frame_bgr
    for each tracklet currently visible:
        draw a dot / box at its position
        # no full mask HUD placement -- no camera calibration exists to place
        # candidate branches at real screen positions

    draw_candidate_hud(out, result)      # side panel listing every scored candidate child
    draw_current_location_label_near_top_of_frame
    return out

function draw_candidate_hud(out, result):
    panel_x, panel_y = fixed HUD corner
    for each (label, candidate_confidence) in result.candidates, sorted by confidence descending:
        draw label name
        draw a horizontal bar sized proportional to fused_confidence
        draw ETA text (from tau_frames if available, else a placeholder dash)
        advance panel_y for next row

```

bronchotrack/fusion/cli.py

Command-line entry point: parses arguments, wires up the detector, tracker, association, and pipeline, then runs the video.

```

function build_arg_parser():
    args: --video, --graph, --weights, --conf-threshold, --out-video, --out-json,

```

```

        --commit-threshold, --commit-frames, --show-graph (mirrors paper_exact's CLI)
    return parser

function main(argv):
    args = build_arg_parser().parse_args(argv)
    graph = AirwayGraph.from_json(args.graph)
    detector = load_detector from args.weights
    tracker = MultiLumenTracker()
    association = FusionAssociation(graph, commit_threshold=args.commit_threshold,
                                   commit_frames=args.commit_frames)
    pipeline = FusionPipeline(detector, tracker, association, graph)

    pipeline.run_on_video(args.video, args.out_video, args.out_json)
    print("Processed <N> frames... Final location: <label> (generation <g>)")

```

bronchotrack/fusion/__init__.py

Package entry point: exports the public classes and documents the three fused models and the deliberate scope reductions for this first version.

```

# Public exports:
from .association import AssociationResult, FusionAssociation
from .kalman_fusion import CandidateConfidence, IdentityFusion
from .motion_model import ApproachEstimate, ApproachMotionModel, TransitSanityChecker
from .pipeline import FrameResult, FusionPipeline

# __all__ = [FusionAssociation, AssociationResult, IdentityFusion, CandidateConfidence,
#            ApproachMotionModel, ApproachEstimate, TransitSanityChecker,
#            FusionPipeline, FrameResult]

# No executable logic beyond imports/exports -- this module is the package's
# documented overview:
# 1. motion_model.ApproachMotionModel -- diameter-growth, scale-free tau
# 2. bronchotrack_id -- point-based bearing matching (segmentation center)
# 3. ratio_id -- whole-mask ratio method (area + boundary-gap, not points)
# kalman_fusion.IdentityFusion blends (1) and (2) into one Hungarian
# assignment per frame; association.FusionAssociation re-runs this every
# frame (continuous re-evaluation) and commits only after a consecutive
# confidence streak.
#
# Deliberate scope reductions for this first version (flagged, not hidden):
# - no roll-angle correction in bronchotrack_id (plain unrotated bearing)
# - no multi-generation "ambiguous cohort" localization smoothing
# - no tracker eligibility dropout (eligibility_fn always True)

```

tests/test_fusion_smoke.py

Smoke and regression tests for the fusion package: geometry primitives, motion model behavior, fusion blending, commit hysteresis, and a no-ML end-to-end run. 18 tests total.

```

helper _rect_mask(cx, cy, half): builds a square polygon mask centered at (cx, cy)
helper _mk_tracklet(track_id, cx, cy, h, mask, label, hits): builds a minimal test Tracklet

test_child_apparent_equivalent_diameter_matches_ellipse_area_formula:
    build a straight branch and a 60-degree-off-axis branch off a common root
    assert straight branch reads ~true diameter; off-axis area-based estimate is
    larger than the off-axis longest-diameter estimate (matches the derived
    cos vs sqrt(cos) foreshortening formulas exactly)
test_mask_boundary_gap_and_circle_boundary_gap_exact:

```

```

    two rectangles separated by a known horizontal gap -> mask_boundary_gap
    returns that exact gap
    two circles at known centers/radii -> circle_boundary_gap returns the exact
    gap; overlapping circles clamp to MIN_DIST

test_ratio_method_disambiguates_where_bearing_alone_cannot:
    construct two candidate branches with identical projected bearing but
    different radius, two detections at the identical image point but
    different mask size
    assert bearing_cost_matrix is completely flat (no discriminating power)
    assert ratio_cost_matrix correctly favors the size-matched pairing

test_ratio_cost_matrix_falls_back_gracefully_without_radius_data:
    graph with no radius data -> ratio_cost_matrix returns None

test_ratio_cost_matrix_handles_no_other_detections_this_frame:
    empty detections list -> ratio_cost_matrix returns None cleanly,
    no NaN / RuntimeWarning

test_bearing_cost_matrix_hungarian_correct_assignment:
    two candidates at right-angle bearings, two detections offset to match each
    assert cost matrix is exactly [[0,1],[1,0]]

test_motion_model_tau_matches_hand_calculation:
    feed linearly growing diameter for 40 frames
    assert estimate.approaching is True and tau_frames is approx
    diameter / growth_rate

test_motion_model_not_approaching_when_shrinking:
    feed shrinking diameter -> approaching is False, tau_frames is None

test_motion_model_prune_drops_stale_filters:
    update with one tracklet, then prune with an empty live set -> filter removed

test_transit_sanity_checker_plausibility:
    no history -> "unknown"
    record two consistent transits -> median speed computed correctly
    a consistent candidate -> "plausible"; a wildly different one -> "implausible"

test_identity_fusion_blends_cues_and_motion_gates:
    two candidates with identical bearing+ratio cost, but only one detection
    is "approaching"
    assert the approaching candidate's fused_confidence ends up higher

test_identity_fusion_falls_back_to_single_cue_when_other_missing:
    bearing_cost = None -> fusion uses ratio_cost alone, unweighted

test_fusion_association_root_bootstrap_labels_both_lumens_at_once:
    two lumens visible at the very first frame, no prior occupant
    assert both get labeled correctly in one frame and the larger becomes
    current_location

test_fusion_association_commits_after_consecutive_frames_not_one_lucky_frame:
    run commit_frames-1 strong frames -> not yet committed
    run one more strong frame -> commits to the correct child

test_fusion_association_streak_resets_on_interrupting_bad_frame:
    4 clearing frames, 1 frame below threshold, 4 more clearing frames
    (commit_frames=5)
    assert it never commits, because the streak resets on the bad frame
    instead of just pausing

test_fusion_association_continuous_reevaluation_survives_reference_loss:

```

```
bootstrap, then a frame with zero tracklets
assert no crash and current_location is unchanged

test_scalar_kalman_filter_promoted_and_reexported_identically:
    assert bronchotrack.scalar_kalman.ScalarKalmanFilter is the same object as
    bronchotrack.paper_exact.scalar_kalman.ScalarKalmanFilter (re-export,
    not a copy)

test_full_fusion_pipeline_smoke_no_ml:
    run tracker + FusionAssociation together on synthetic carina-region
    detections (no real ML)
    assert both main bronchi get bootstrapped and labeled correctly

_run_all(): runs every test above in sequence, prints PASS/FAIL per test and
a final N/M summary
```